

TRABALHO PRÁTICO 4

CityDrive

Sistema de Reserva de Viaturas

Formanda	Esmeralda Fonseca
Professor	Eng.º Sebilson Cristóvão
Função	Project Manager
Tecnologias	Python · Flet · MySQL · Git
Data de entrega	01 de outubro de 2026

ENUNCIADO

1. Enquadramento

No desenvolvimento profissional de software, um programador raramente inicia todos os projetos a partir de uma pasta vazia. É frequente integrar uma equipa e receber uma aplicação já iniciada por outro programador, sendo necessário compreender a arquitetura existente, interpretar código desenvolvido por terceiros, identificar funcionalidades incompletas e continuar o desenvolvimento sem comprometer a estrutura do sistema.

O Trabalho Prático 4 foi concebido precisamente com esse objetivo. Ao contrário de trabalhos em que a solução é construída de raiz, neste projeto é fornecida à formanda Esmeralda Fonseca uma aplicação parcialmente desenvolvida, com modelos, acesso à base de dados, Services, Repositories, Views em Flet e diversos pontos de implementação identificados através de TODO.

O objetivo não consiste em reconstruir a aplicação. A formanda deverá estudar, compreender, completar e testar o projeto fornecido, respeitando as decisões arquiteturais existentes e demonstrando capacidade para trabalhar sobre código criado por terceiros.

Contexto

profissional

O projeto simula uma situação em que o Project Manager, Eng.º Sebilson Cristóvão, entrega à programadora uma base de código previamente estruturada, define os requisitos e acompanha tecnicamente a evolução da implementação.

2. Cenário

A empresa fictícia CityDrive disponibiliza viaturas para reserva em diferentes cidades. Um cliente que chegue, por exemplo, a Castelo Branco, poderá utilizar a aplicação para indicar a cidade onde pretende levantar a viatura, a data e hora de início da reserva e a data e hora de fim.

A aplicação deverá consultar as viaturas existentes e apresentar apenas aquelas que se encontram disponíveis para o período solicitado. Para cada viatura, o cliente deverá conseguir consultar informação relevante sobre o veículo e sobre a estação onde este se encontra, podendo posteriormente confirmar a reserva.

3. Exemplo de situação profissional

O Project Manager, Eng.º Sebilson Cristóvão, necessita de se deslocar a Castelo Branco para acompanhar presencialmente uma fase importante de um projeto. Pretende reservar uma viatura entre as 12:00 e as 15:00 do dia 10 de setembro de 2026.

Uma determinada viatura encontra-se reservada anteriormente entre as 06:30 e as 12:00. A nova reserva deverá ser permitida, uma vez que a reserva anterior termina exatamente às 12:00.

Reserva existente: 06:30 ----- 12:00

Nova reserva: 12:00 ----- 15:00

Resultado: PERMITIDA

Se, pelo contrário, o Project Manager necessitasse da mesma viatura a partir das 11:30, existiria sobreposição entre as reservas e o pedido deveria ser rejeitado.

```
Reserva existente: 06:30 ----- 12:00
Nova reserva:      11:30 ----- 15:00
Resultado: NÃO PERMITIDA
```

4. Objetivos do Trabalho

Com este trabalho pretende-se consolidar e avaliar competências de programação e desenvolvimento de software, com especial incidência na leitura e manutenção de um projeto existente.

- Interpretação de código desenvolvido por terceiros e orientação dentro de um projeto com vários módulos.
- Programação Orientada a Objetos e arquitetura por camadas.
- Desenvolvimento de interfaces gráficas com Flet.
- Utilização de MySQL, chaves primárias, chaves estrangeiras, consultas SQL e JOIN.
- Implementação de operações CRUD e tratamento de datas e horas.
- Validação de dados, regras de negócio e deteção de conflitos temporais.
- Tratamento de erros, testes, qualidade de código e utilização adequada de Git.
- Capacidade de explicar tecnicamente as soluções implementadas.

5. Material fornecido

A formanda recebe um projeto Python previamente estruturado, incluindo os principais elementos apresentados abaixo.

```
CityDrive/
|
|-- main.py
|-- database.py
|-- models/
|-- repositories/
|-- services/
|-- views/
|-- database/
|-- docs/
|-- requirements.txt
`-- README.md
```

É igualmente fornecido o script `database/citydrive.sql`, responsável pela criação da estrutura inicial da base de dados MySQL. A base de dados constitui um esqueleto estrutural e não uma base de dados previamente preenchida com todos os dados necessários à aplicação.

6. Regra fundamental do Trabalho Prático 4

A formanda não deverá começar imediatamente a programar os TODO. Antes de qualquer implementação deverá compreender a estrutura recebida e o fluxo das operações.

1. Executar o projeto e confirmar o seu estado inicial.
2. Ler o README e a documentação fornecida.
3. Analisar a estrutura de pastas e identificar as responsabilidades das camadas.
4. Analisar Models, Repositories, Services, Views e a base de dados.
5. Ler as regras de negócio.
6. Localizar e interpretar os TODO.
7. Implementar progressivamente as funcionalidades.
8. Testar situações normais, inválidas e situações limite.

7. Arquitetura obrigatória

O projeto encontra-se organizado segundo uma arquitetura em camadas, que deverá ser respeitada durante todo o desenvolvimento.

Flet View

|

v

Service

|

v

Repository

|

v

Database

|

v

MySQL

As Views são responsáveis pela interface e interação com o utilizador. Os Services concentram validações e regras de negócio. Os Repositories são responsáveis pelo acesso aos dados e pelas instruções SQL. A classe Database centraliza a ligação ao MySQL.

Regra

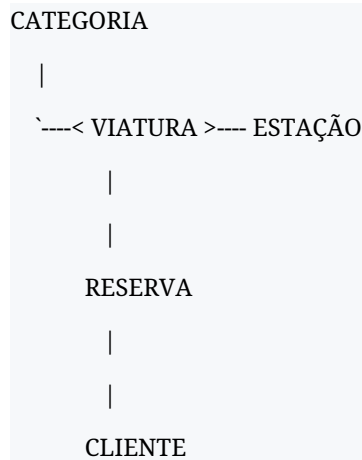
técnica

obrigatória

Uma View não deverá executar SQL diretamente. Do mesmo modo, um Repository não deverá construir interfaces Flet nem concentrar regras de negócio que pertençam aos Services.

8. Entidades e base de dados

A aplicação trabalha com cinco entidades principais: Cliente, Categoria, Estação, Viatura e Reserva. A formanda deverá compreender os respetivos atributos e as relações existentes entre elas.



A base de dados MySQL contempla as tabelas clientes, estacoes, categorias_viatura, viaturas e reservas. A formanda deverá analisar tipos de dados, chaves primárias, chaves estrangeiras, restrições, índices e relações entre tabelas.

9. Requisitos funcionais

9.1 Clientes

Um cliente deverá possuir identificador, nome, email e telefone. O email deverá permitir identificar um cliente já existente. Na entrada da aplicação, o sistema deverá verificar os dados introduzidos e utilizar os Services e Repositories adequados.

9.2 Estações

Cada estação CityDrive deverá possuir nome, morada, cidade, hora de abertura, hora de fecho e estado ativo/inativo. Uma estação inativa não deverá ser considerada numa pesquisa normal de disponibilidade. O horário deverá ser considerado durante o processo de reserva.

9.3 Categorias

As viaturas deverão estar associadas a uma categoria. A categoria possui informação própria e deverá ser relacionada com a viatura através da base de dados. A aplicação deverá obter esta informação através das relações existentes entre as tabelas.

9.4 Viaturas

Cada viatura deverá possuir matrícula, marca, modelo, categoria, ano, estação e estado ativo/inativo. Uma viatura inativa não poderá ser apresentada como disponível para uma nova reserva. A matrícula deverá identificar unicamente uma viatura.

9.5 Pesquisa

O cliente deverá conseguir pesquisar viaturas indicando a cidade, a data/hora de início e a data/hora de fim. O sistema deverá apresentar apenas as viaturas que satisfaçam as regras de negócio, e não a lista completa de viaturas existentes.

Cidade: Castelo Branco

Início: 2026-09-10 12:00

Fim: 2026-09-10 15:00

Nos resultados e/ou no detalhe da viatura deverão ser disponibilizadas informações relevantes, como marca, modelo, matrícula, ano, categoria, estação, cidade e horário da estação.

10. Regra temporal fundamental

Uma reserva ocupa a viatura durante o intervalo compreendido entre o seu início e o seu fim. O instante correspondente ao fim deixa de pertencer ao período de ocupação. Conceptualmente, o intervalo deve ser interpretado como [início, fim).

Reserva existente: 06:30 ----- 12:00

08:00 --- 09:00 -> conflito

10:00 ----- 14:00 -> conflito

12:00 ----- 15:00 -> permitido

05:00 ----- 06:30 -> permitido, se o horário da estação o permitir

A solução deverá ser geral e não baseada em valores específicos utilizados nos exemplos.

11. Conflitos entre reservas

A aplicação deverá detetar corretamente diferentes formas de sobreposição. Existindo uma reserva das 10:00 às 14:00, deverão ser considerados conflito todos os pedidos que atravessem, estejam contidos ou contenham esse intervalo.

Pedido	Resultado
11:00 - 12:00	Rejeitar
09:00 - 11:00	Rejeitar
13:00 - 15:00	Rejeitar
09:00 - 16:00	Rejeitar
10:00 - 14:00	Rejeitar
08:00 - 10:00	Permitir
14:00 - 16:00	Permitir

12. Horário das estações

Uma reserva deverá respeitar o horário da estação associada à viatura. Se uma estação abrir às 06:00 e fechar às 22:00, um levantamento às 05:30 ou uma devolução às 23:00 deverão ser rejeitados.

Uma reserva poderá abranger mais do que um dia. Nesses casos, a hora prevista para o levantamento e a hora prevista para a devolução deverão respeitar o horário da estação.

13. Estados das reservas

O projeto contempla os estados PENDENTE, CONFIRMADA, EM_CURSO, CONCLUIDA e CANCELADA. A formanda deverá compreender a finalidade destes estados e utilizá-los de forma coerente.

Disponibilidade

Reservas CANCELADA e CONCLUIDA não deverão bloquear a viatura numa nova pesquisa de disponibilidade.

14. Criação e cancelamento de reservas

Antes de criar uma reserva, o sistema deverá confirmar as condições necessárias. A operação deverá validar o período, o cliente, a viatura, a estação, o horário e a inexistência de conflitos.

Pedido de reserva

|

v

Período válido?

|

Cliente válido?

|

Viatura existe e está ativa?

|

Estação existe e está ativa?

|

Horário válido?

|

Existe conflito?

|

NÃO

|

v

Criar reserva

Caso uma das condições não seja satisfeita, a operação deverá ser impedida e a aplicação deverá apresentar uma mensagem adequada. O cancelamento deverá utilizar o estado CANCELADA, preservando o histórico, em vez de eliminar fisicamente a reserva apenas para a fazer desaparecer.

15. Área do cliente e área administrativa

A área 'As minhas reservas' deverá permitir ao cliente identificar viatura, estação, início, fim e estado de cada reserva. Quando aplicável, deverá também disponibilizar o cancelamento.

A área administrativa deverá permitir trabalhar informação relacionada com estações, viaturas e reservas, utilizando a mesma arquitetura da restante aplicação. A implementação de autenticação administrativa avançada não constitui o objetivo principal deste trabalho, salvo indicação posterior do professor.

16. TODO obrigatórios

O projeto fornecido contém 31 TODO numerados, organizados por blocos de desenvolvimento. A numeração existe para orientar uma implementação progressiva.

Bloco	TODO	Área principal
A	1 - 7	Categorias e Estações
B	8 - 13	Viaturas e JOIN
C	14 - 19	Reservas e conflitos temporais
D	20 - 23	Regras de negócio
E	24 - 28	Integração com Flet
F	29 - 31	Administração

17. Tratamento de erros e qualidade do código

A aplicação não deverá terminar abruptamente perante situações previsíveis. Deverão ser tratados adequadamente campos obrigatórios vazios, datas inválidas, data final anterior à inicial, entidades inexistentes, estados inativos, períodos fora do horário da estação, conflitos de reservas e problemas durante operações na base de dados.

O código deverá ser organizado, legível, modular e coerente com o código fornecido. Comentários deverão ser utilizados quando acrescentarem informação útil e não para explicar instruções triviais.

18. Alteração da arquitetura

A arquitetura fornecida faz parte da avaliação. Não deverão ser eliminadas camadas para simplificar o trabalho, transferidas queries SQL para Views, colocadas regras de negócio complexas nos Repositories, concentradas todas as funcionalidades num único ficheiro ou substituídas as tecnologias definidas sem autorização.

Alterações

estruturais

Caso a formanda considere necessária uma alteração estrutural, deverá apresentar previamente a respetiva justificação técnica ao Project Manager, Eng.º Sebilson Cristóvão. A autorização de uma alteração não é automática apenas porque a solução alternativa funciona.

19. Git e evolução do projeto

O desenvolvimento deverá demonstrar evolução progressiva. Não é adequado desenvolver todo o trabalho e efetuar apenas um commit final. São esperados commits associados a etapas concretas e com mensagens que permitam perceber a evolução real do projeto.

Implementação das consultas de estações
 Implementação do repository de viaturas
 Implementação da deteção de conflitos
 Validação das regras de reserva
 Integração da pesquisa com Flet
 Implementação da área de reservas do cliente

20. Testes obrigatórios

A formanda deverá testar o sistema antes da entrega. Entre os testes mínimos encontram-se os seguintes.

Situação	Resultado esperado
Início anterior ao fim	Aceitar validação temporal
Início igual ao fim	Rejeitar
Início posterior ao fim	Rejeitar
Viatura inativa	Não disponibilizar
Estação inativa	Não disponibilizar
Reserva dentro de outra reserva	Rejeitar
Reserva que contém outra reserva	Rejeitar
Sobreposição no início	Rejeitar
Sobreposição no fim	Rejeitar
Nova reserva começa quando outra termina	Permitir
Nova reserva termina quando outra começa	Permitir
Reserva fora do horário da estação	Rejeitar
Reserva cancelada existente	Não bloquear a viatura

21. Casos de referência do professor

Durante a avaliação, o professor poderá criar dados e períodos diferentes daqueles utilizados pela formanda durante o desenvolvimento. O sistema deverá continuar a produzir o resultado correto, independentemente dos valores utilizados.

Por exemplo, o Eng.º Sebilson Cristóvão poderá solicitar uma viatura numa estação de Castelo Branco num determinado intervalo e, imediatamente depois, efetuar outra pesquisa que coincida apenas parcialmente com esse intervalo. A aplicação deverá aplicar corretamente as regras de disponibilidade.

Importante

Os exemplos deste enunciado servem para explicar regras. Não deverão ser utilizados como valores fixos no código.

22. Valores fixos e soluções simuladas

Não deverão ser utilizados valores artificiais para simular funcionalidades concluídas. Um `cliente_id` fixo, listas de viaturas escritas manualmente numa View ou estados definidos apenas para produzir determinado resultado não substituem a implementação correta.

A informação deverá percorrer corretamente as diferentes camadas da aplicação.

23. Defesa individual do trabalho

A entrega do código não constitui, por si só, prova suficiente da aquisição das competências avaliadas. A formanda deverá conseguir explicar o código apresentado e relacionar cada operação com a arquitetura do projeto.

Durante a defesa, o Eng.º Sebilson Cristóvão poderá solicitar que a formanda identifique o ponto de entrada da aplicação, explique o percurso de uma operação entre as diferentes camadas, interprete uma query ou JOIN, explique a deteção de conflitos, localize uma regra de negócio, altere uma pequena parte do código ou analise um caso de teste.

24. Exemplo de questão de defesa

Questão

possível

O Eng.º Sebilson Cristóvão pretende reservar uma viatura das 14:00 às 17:00. A mesma viatura possui uma reserva confirmada das 12:00 às 14:00. O sistema deve aceitar ou rejeitar a nova reserva? Em que parte do projeto é tomada essa decisão e que camadas participam na operação?

Não será suficiente responder apenas que a operação é aceite. A formanda deverá explicar porquê e relacionar a resposta com a implementação realizada, reconhecendo que uma nova reserva pode começar exatamente no instante em que termina a anterior.

25. Capacidade de manutenção

O professor poderá solicitar uma pequena alteração não prevista literalmente nos TODO, com o objetivo de verificar se a formanda compreendeu suficientemente o projeto para localizar o ponto correto de alteração.

Exemplo: o Project Manager pretende que, no detalhe da viatura, passe também a ser apresentado o telefone da estação, assumindo que esse campo passou a existir na base de dados. A formanda deverá ser capaz de identificar as partes do projeto que teriam de ser analisadas ou alteradas.

26. Utilização de código não compreendido

Todo o código entregue poderá ser objeto de questões durante a defesa. A formanda assume responsabilidade técnica pelo código existente na sua versão final do projeto.

Caso seja apresentada uma implementação que a formanda não consiga explicar, reproduzir conceptualmente ou relacionar com o restante projeto, essa componente poderá não ser considerada como competência demonstrada.

27. Entrega

A entrega deverá incluir o código-fonte completo, o script SQL, o requirements.txt, a documentação solicitada e o histórico Git do desenvolvimento. Não deverão ser incluídas pastas ou ficheiros desnecessários, nomeadamente o ambiente virtual .venv.

Data	de	entrega
01 de outubro de 2026		

28. Critérios gerais de avaliação

Área	Peso indicativo
Compreensão da arquitetura e código fornecido	15%
MySQL, Repositories e consultas SQL	20%
Regras de negócio e Services	20%
Disponibilidade e conflitos de reservas	15%
Integração e funcionamento da interface Flet	10%
Área administrativa e funcionalidades complementares	5%
Tratamento de erros, testes e qualidade do código	5%
Git e organização do desenvolvimento	5%
Defesa individual e domínio técnico	5%
TOTAL	100%

A distribuição poderá ser ajustada pelo professor em função da evolução do trabalho e das competências que necessitem de maior incidência avaliativa.

29. Penalizações técnicas

Poderão implicar perda significativa de cotação: não respeitar a arquitetura fornecida, colocar SQL diretamente nas Views, substituir funcionalidades por dados fixos, remover TODO sem implementar a funcionalidade, não validar conflitos entre reservas, ignorar horários das estações, apresentar uma aplicação que termina perante erros previsíveis, efetuar alterações estruturais sem justificação ou não conseguir explicar componentes relevantes da solução entregue.

30. Resultado final esperado

No final do Trabalho Prático 4, espera-se uma aplicação capaz de executar de forma coerente o fluxo apresentado abaixo.

CLIENTE

|

v

Entrada / identificação

|

v

Escolha da cidade e do período

|

v

Pesquisa

|

v

VIATURAS DISPONÍVEIS

|

v

Detalhe da viatura e estação

|

v

Confirmação

|

v

VALIDAÇÕES

|

v

Reserva criada

|

v

As minhas reservas

31. Competência central avaliada

Este trabalho representa uma evolução relativamente aos exercícios em que a solução é iniciada do zero. A principal questão deixa de ser apenas se a formanda consegue programar determinada funcionalidade e passa também a avaliar se consegue compreender uma aplicação existente, identificar onde uma funcionalidade deve ser implementada e continuar o desenvolvimento respeitando o trabalho já realizado.

32. Orientação final à formanda

Antes de escrever código, compreenda o projeto. Antes de alterar um ficheiro, identifique a responsabilidade desse ficheiro. Antes de escrever SQL, confirme que está na camada correta. Antes de criar uma regra numa View, verifique se essa regra pertence a um Service. Antes de considerar um TODO concluído, teste situações válidas, inválidas e situações limite.

Regra	final
Todo o código entregue deverá ser compreendido e tecnicamente defendido pela formanda.	

Formanda: Esmeralda Fonseca

Professor: Eng.º Sebilson Cristóvão | Project Manager

Data de entrega: 01 de outubro de 2026